Evolutionary Games Toolkit
v. 2025.09.29
Documentation

The GameTheory2025 Collective

October 8, 2025

0 Introduction

The *Evolutionary Games Toolkit* (EGT) is a Matlab toolbox aimed at the study of (obviously) evolutionary games. Our definition of what *is* an "evolutionary game" is given in Section 2. EGT provides functions for three types of operations:

1. Simulation.
2. Markovian analysis.
3. Mean Field Dynamics (MFD) simulation.

The exact meaning of the above terms is explained in the following sections.

1 Quick Start

EGT has been created with Matlab 2023b; while we have not tested it, it probably works with every Matlab version after 2020. It can be obtained from `https://github.com/thanasiskehagias/EvolutionaryGamesToolkit` and also from the Matlab File Exchange. It is provided under the MIT license.

To start using EGT, unzip `EGT.zip` and you will obtain a root folder `EGT` with several subfolders. Open Matlab, go to `EGT` and first run the script `AddMyPath`. Next go to subfolder `Examples` and then to any of its subfolders and run any of the scripts included in these; each script produces some plots which show the evolution of strategy usage over several generations of an evolutionary game.

2 Evolutionary Games and Markov Chains

Classical descriptions of evolutionary games appear in [2, 5, 6]. Our main starting point is [6], but we use a simplified version of the model presented therein. We find that our results closely approximate those of [6].

In our formulation, the ingredients of an evolutionary game are the following. We start with a *symmetric* two-player game described by an $M \times M$ bimatrix $\left(B, B^T\right)$; i.e., each player can use a pure strategy from the set $\Sigma = \{\sigma_1, ..., \sigma_M\}$ and we have

$$B_{m_1 m_2} = \text{"the payoff received by row player when he uses } \sigma_{m_1} \text{ against column player using } \sigma_{m_2}\text{"}.$$

Now assume the existence of N *players* and define the following:

1. A *population* vector $\mathbf{z} = (z_1, ..., z_N) \in \mathbf{Z}$ indicates the strategy used by each player, i.e., $z_n = m$ means the n-th player is using the m-th strategy; we will often call such a player an "*m-user*". It takes values in the

$$\text{Population Space: } \mathbf{Z} = \{(z_1, ..., z_N) : \forall n : z_n \in \{1, .., M\}\}.$$

2. A *strategy* or *state* vector $\mathbf{s} = (s_1, ..., s_M) \in \mathbf{S}$ indicates how many players are using each strategy, i.e., s_m is the number of m-users. It takes values in the

$$\text{State Space: } \mathbf{S} = \left\{ (s_1, ..., s_M) \in \{0, 1, .., N\} \text{ such that } \sum_{m=1}^{M} s_m = N \right\}.$$

3. A *frequencies* vector $\mathbf{x} = (x_1, ..., x_M) \in \mathbf{\Delta}_M$ (the M-dimensional unit simplex) where $x_m = \frac{s_m}{N}$ indicates the proportion of m-users in the population.

For given payoff matrix B, and number of players N, the evolutionary game consists of the following procedure.

Evolutionary Game Protocol

1. First we choose an initial population vector $\mathbf{z}(0) = (z_1(0), ..., z_N(0)) \in \mathbf{Z}$, i.e., an initial distribution of strategies to players.
2. Then we repeat for $j \in \{0, 1, ..., J-1\}$ the following iteration.
   (a) We compute the state vector $\mathbf{s}(j) = (s_1(j), ..., s_M(j)) \in \mathbf{S}$ and the *frequency vector* $\mathbf{x}(j) = (x_1(j), ..., x_M(j))$ as follows:

$$\forall m : s_m(j) = |\{z_n(j) : z_n(j) = m\}|, \qquad x_m(j) = \frac{s_m(j)}{\sum_{m=1}^{M} s_m(j)}.$$

   (b) The players play all possible matches between each other, i.e., for all the pairs (n_1, n_2) with $n_1, n_2 \in \{1, ..., N\}$ and $n_1 \neq n_2$. In each match they play one round of the B game.
   (c) The n-th player (for $n \in \{1, ..., N\}$) collects a total payoff

$$q_n(j) = \sum_{n \neq n'} B_{z_n(j) z_{n'(j)}},$$

   i.e., the n-th player collects his total payoff by playing strategy z_n against the n'-th player's strategy $z_{n'}$, for every $n' \neq n$.
   (d) The m-th strategy collects a total payoff

$$Q_m(j) = \sum_{n : z_n = m} q_n(j),$$

   i.e., the total payoff of the m-th strategy is the sum of the payoffs of the m-users.
   (e) We obtain the next population vector $\mathbf{z}(j+1)$ (also called a *generation*) by letting some players update their strategy. This is achieved by a *revision protocol* $\mathcal{R}$; we will presently discuss several such protocols.

So an evolutionary game is a pair $(B, \mathcal{R})$ (with auxiliary parameters N and J) and produces vector sequences $(\mathbf{z}(j))_{j=0}^{\infty}$, $(\mathbf{s}(j))_{j=0}^{\infty}$ and $(\mathbf{x}(j))_{j=0}^{\infty}$. It is clear that the population process $(\mathbf{z}(j))_{j=0}^{\infty}$ is a Markov chain and a little thought shows that the same is true of the state process $(\mathbf{s}(j))_{j=0}^{\infty}$; the *frequency process* $(\mathbf{x}(j))_{j=0}^{\infty}$ is simply a function of $(\mathbf{s}(j))_{j=0}^{\infty}$.

Given a specific evolutionary game $(B, \mathcal{R})$, we want to study the long term behavior of $(\mathbf{s}(j))_{j=1}^{\infty}$. By choosing revision protocols which favor better performing strategies (i.e., those with higher total payoff), we hope that $(\mathbf{s}(j))_{j=0}^{\infty}$ will converge into a state $\overline{\mathbf{s}}$ in which all players are using the best performing strategy (more accurately: *one* the best performing strategies).

We can classify revision protocols by the number of players who may change their strategy (in one generation). A *K-revision protocol* is one in which K players can change. In EGT we have only implemented 1- and N-revision protocols.[1] Let us now present these.

We will use the following notations. We write the player total payoff vector as $\mathbf{q} = (q_1, ..., q_N)$ and the strategy total payoff vector as $\mathbf{Q} = (Q_1, ..., Q_M)$. Note that both $\mathbf{q}$ and $\mathbf{Q}$ are functions of the state vector $\mathbf{s}$ (or, equivalently, the frequency vector $\mathbf{x}$), the payoff matrix B etc. However, for the sake of brevity, we will usually omit this dependence from our notation. We let $\overline{Q}$ be the average strategy payoff and $\widehat{Q}$ be the maximum strategy payoff

$$\overline{Q} = \sum_{m=1}^{M} x_m Q_m, \quad \widehat{Q} = \max_m Q_m.$$

Also we use the notation $[y]_+ = \max(y, 0)$, i.e., $[y]_+$ is the nonnegative part of y. Finally, $\mathbf{1}_{\mathbf{U}}(u)$ is the indicator function of set $\mathbf{U}$, i.e., it equals 1 when the $u \in \mathbf{U}$ and 0 when $u \notin \mathbf{U}$.

EGT contains a single N-revision protocol, the **Simultaneous Proportional Revision**, which works as follows. In each generation *all* players update their strategies simultaneously; the n-th player adopts the m-th strategy with probability:

$$\forall m \in \{1, ..., M\}) : \pi_m = \frac{\sum_{n:s_n=m} q_n(j)}{\sum_{n=1}^{N} q_n(j)}.$$

EGT contains several 1-revision protocols, all of which follow the same general pattern:

1. A player is selected equiprobably from the population.
2. If the player is an m_1-user, then he adopts the m_2-th strategy with probability $R_{m_1 m_2}(\mathbf{s})$, where $R(\mathbf{s})$ is an $M \times M$ *rate matrix* depending on the current state $\mathbf{s}$. By specifying $R(\mathbf{s})$ we obtain a specific 1-revision protocol.

EGT contains the following rate matrix implementations (for brevity of notation, we omit the dependence on $\mathbf{s}$).

1. **Best Response**. Strategy m_1 switches equiprobably to the strategy of one of the best performing players. I.e., letting

$$\text{the set of the players who achieve max payoff: } \widehat{\mathbf{N}} = \left\{ n : q_n = \max_i q_i \right\},$$
$$\text{the set of the strategies of these players: } \widehat{\mathbf{M}} = \left\{ z_n : n \in \widehat{\mathbf{N}} \right\},$$

   we set

$$R_{m_1 m_2} = \frac{\mathbf{1}_{\widehat{\mathbf{M}}}(m_2)}{\left|\widehat{\mathbf{M}}\right|}.$$

2. **Pairwise Proportional Comparison**. Strategy m_1 switches to m_2 with probability proportional to the positive part of the surplus of Q_{m_2} over Q_{m_1}, multiplied by the frequency of m_2:

$$R_{m_1 m_2} = \frac{x_{m_2} [Q_{m_2} - Q_{m_1}]_+}{\sum_{m=1}^{M} x_m [Q_m - Q_{m_1}]_+}.$$

3. **Pairwise Comparison**. Strategy m_1 switches to m_2 with probability proportional to the positive part of the surplus of Q_{m_2} over Q_{m_1}:

$$R_{m_1 m_2} = \frac{[Q_{m_2} - Q_{m_1}]_+}{\sum_{m=1}^{M} [Q_m - Q_{m_1}]_+}.$$

[1] It should be fairly easy for the user to modify our code to produce K-revision protocols.

4. **Comparison to the Average Payoff**. Strategy m_1 switches to m_2 with probability proportional to positive part of the Q_{m_2} surplus over $\overline{Q}$:

$$R_{m_1 m_2} = \frac{\left[Q_{m_2} - \overline{Q}\right]_+}{\sum_{m=1}^{M} \left[Q_m - \overline{Q}\right]_+}.$$

5. **Logit**. Strategy m_1 switches to m_2 with the following probability:

$$R_{m_1 m_2} = \frac{\exp\left(Q_{m_2}/\eta\right)}{\sum_{m=1}^{M} \exp\left(Q_m/\eta\right)}$$

where η is a revision parameter.

It is worth pointing out that all of the above rate matrices assign zero probability to the adoption of a strategy which is not represented in the population (why?). In other words, once a strategy becomes extinct in an evolutionary game, it will never reappear. Having selected a rate matrix R, we can compute the transition probability matrix P as follows[2]

$$\forall \mathbf{s}, \mathbf{s}' : P_{\mathbf{s}\mathbf{s}'} = \Pr\left(\mathbf{s}\left(j+1\right) = \mathbf{s}' | \mathbf{s}\left(j\right) = \mathbf{s}\right) = \begin{cases} x\left(\mathbf{s}\right) R_{m_1 m_2}\left(\mathbf{s}\right) & \text{if } s'_{m_1} = s_{m_1} - 1 \text{ and } s'_{m_2} = s_{m_2} + 1 \\ 0 & \text{else} \end{cases} \tag{2.1}$$

We can use the transition probability matrix P , as given by (2.1), in two ways.

1. We can generate an instance of the Markov process $\{\mathbf{s}\left(j\right)\}_{j=0}^{\infty}$, where each $\mathbf{s}\left(j+1\right)$ is generated from the (pre-computed) $\Pr\left(\mathbf{s}\left(j+1\right)|\mathbf{s}\left(j\right)\right)$ for a given $\mathbf{s}\left(j\right)$. We call this *Markovian simulation*; note that this is different from "plain" simulation, in which we simply implement the Evolutionary Game Protocol.

2. In addition, for $M = 3$ strategies we can use P to produce a visual *state transition diagram*. In such a diagram every state $\mathbf{s} = (s_1, s_2, s_3)$ is denoted by a circle with coordinates (s_1, s_2) [3] and, for every state $\mathbf{s}' = (s'_1, s'_2, s'_3)$ for which $P_{\mathbf{s},\mathbf{s}'} > 0$ (i.e., a transition from $\mathbf{s}$ to $\mathbf{s}'$ is possible) we add an arrow $\mathbf{s}$ to $\mathbf{s}'$. Examples of this representation will be presented in Section 5.

3 Mean Field Dynamics

It is well known [5, 6] that, for certain revision protocols, the evolution of the frequency process $(\mathbf{x}\left(j\right))_{j=1}^{\infty}$ can be approximated, for large N, by a *mean field dynamic* (MFD) expressed as a system of ordinary differential equations. The following table summarizes some of these correspondences.

Revision Protocol	**Mean Field Dynamic**
Simultaneous Proportional Revision	Type 2 Replicator Dynamics
Pairwise Proportional Comparison	Type 1 Replicator Dynamics
Pairwise Comparison	Smith Dynamics
Comparison to Average Payoff	Brown-von Neumann-Nash Dynamics
Logit	Logit Dynamics

Table 1: Correspondence between revision protocols and mean field dynamics.

In what follows we will use the auxiliary quantities:

$$\text{Average Payoff:} \quad \overline{Q}\left(\mathbf{x}\right) = \sum x_i Q_i\left(\mathbf{x}\right)$$

$$m\text{-th Strategy Excess Payoff:} \quad \widehat{Q}_m\left(\mathbf{x}\right) = Q_m\left(\mathbf{x}\right) - \overline{Q}\left(\mathbf{x}\right)$$

Now we can present the list of dynamics.

[2] Actually, the above formula holds only for "interior" states, i.e., those with $s_m \in \{2, ..., N-1\}$ for all m. The modifications for "boundary" states are straightforward; for example, with $N = 3$: the only possible transition from $(3, 0, 0)$ is to itself, the possible transitions from $(2, 1, 0)$ are to $(3, 0, 0)$ and to $(1, 2, 0)$ etc.

[3] No informaiton is lost by this representation, since we will always have $s_3 = N - s_1 - s_2$.

1. **Replicator Dynamics Type 2** (discrete time).

$$\forall j, m : x_m (j+1) = \frac{\sum_{n:s_n=m} q_n (j)}{\sum_{n=1}^{N} q_n (j)}.$$

2. **Replicator Dynamics Type 1** (continuous time).

$$\forall m : \frac{dx_m}{dt} = x_m (t) (Q_m (\mathbf{x}) - Q (\mathbf{x})).$$

3. **Smith Dynamics** (continuous time).

$$\forall m : \frac{dx_m}{dt} = \sum_{i=1}^{M} x_i [Q_m (\mathbf{x}) - Q_i (\mathbf{x})]_+ - x_m \sum_{i=1}^{M} [Q_i (\mathbf{x}) - Q_m (\mathbf{x})]_+ .$$

4. **Brown-von Neumann-Nash Dynamics** (continuous time).

$$\forall m : \frac{dx_m}{dt} = \left[\widehat{Q}_m (\mathbf{x})\right]_+ - x_m \sum_{i=1}^{M} \left[\widehat{Q}_i (\mathbf{x})\right]_+ .$$

5. **Logit Dynamics** (continuous time).

$$\forall m : \frac{dx_m}{dt} = \frac{\exp\left(\frac{Q_m(\mathbf{x})}{\eta}\right)}{\sum_{i=1}^{M} \exp\left(\frac{Q_i(\mathbf{x})}{\eta}\right)}.$$

   η is a "noise parameter" used in the logit dynamics, with small values resulting in faster convergence.

4 The Basic EGT Functions

EGT contains five basic functions:

1. `[P,S]=EGTMarkP(B,Dynamic)` computes the transition probability matrix of an evolutionary game.
2. `[s,x]=EGTMarkSim(P,S,s0,J)` simulates an evolutionary game using the transition probability matrix (Markovian simulation).
3. `[s,x,t]=EGTMFDyn(B,Dynamic,s0,J)` computes the the continuous time mean field dynamics of an evolutionary game.
4. `[s,x]=EGTMFDynDisc(B,Dynamic,s0,J)` computes the discrete time mean field dynamics of an evolutionary game.
5. `[s,x]=EGTSim(B,Dynamic,s0,J)` simulates an evolutionary game by implementing the Evolutionary Game Protocol (plain simulation).

The input variables are as follows.

`B`	:	Payoff matrix
`Dynamic`	:	Dynamic or Protocol
`P`	:	Transition probability matrix
`S`	:	Srategy state space matrix
`s0`	:	Initial state
`J`	:	Number of generations

In addition the following variables essentially work as input to the main functions, but must be declared as global (for more details see the script listings in the Examples section).

`B`	:	Payoff matrix
`M`	:	Number of strategies
`N`	:	Number of players
`eta`	:	Noise parameter (only used by Logit functions)

The output variables are as follows.

`P`	:	Transition probability matrix
`S`	:	Srategy state space matrix
`s`	:	Strategy state sequence (across generations)
`x`	:	Frequency sequence (across generations)
`t`	:	Times sequence

The variable `t` is needed because the Matlab ODE solvers do not always output the vector of $x(t)$ values at equispaced time steps.

A full list of the `EGT` functions, with some explanation of their use, can be found in Section A of the Appendix. More details are provided as comments in the corresponding function files.

5 Examples

5.1 Prisoner's Dilemma

In this section we present in some detail at the application of EGT on an evolutionary game which uses as base game the standard Prisoner's Dilemma (PD). This has the the payoff matrix

$$A = \begin{bmatrix} 3 & 1 \\ 4 & 2 \end{bmatrix}$$

The required Matlab scripts are `PD_MFD01.m` and `PD_Sim01.m`, contained in the folder `Examples/Basic`. Let us look at `PD_MFD.m`.

```
1  clear; clc;
2  global B M eta
3  B=[3 1; 4 2];                                          % Prisoner's Dilemma
4  Strategies = ["Cooperate","Defect"];
5  M=length(Strategies);
6  s0=[12 8];
7
8  J=20;
9  ExpName='PD_MFDRep1';GrName='Replicator 1 MF Dynamics';
10 [s,x,t]= EGTMFDyn(B,'Rep1',s0,J);
11 GrDynX(s,x,t,Strategies,ExpName,GrName);
12
13 J=200;
14 ExpName='PD_SimRep1';GrName='Replicator Type 1 Simulation';
15 [s,x]=EGTSim1(B,'Rep1',s0,J);
16 GrSim(s,x,Strategies,ExpName,GrName);
```

The following remarks explain the operation of the script.

1. In line 1 we clear the workspace.
2. In line 2 we declare the variables `B` (payoff matrix), `M` (number of strategies) and `eta` (a logit dynamics parameter) as global. This must be *always* done, as these variables are required for the Matlab ODE solver to simulate the evolutionary dynamics.

3. In lines 4-5 we initialize the `Strategies` vector (the strategies to be used are ' 'Cooperate" and "Defect") and the `M` variable (the number of strategies equals the length of the strategies vector).

4. In line 6 we specify the initial distribution of strategies: we will use 12 cooperators and 8 defectors.

5. In lines 8-11 we run the Mean Field Dynamics.

   (a) Line 8: we will integrate the evolutionary dynamics from $t_0 = 0$ to $t_1 =$`J`, which we take equal to 20.

   (b) Line 9: the `ExpName` is used to store the plot of the frequency vector evolution and the `GrName` will be the title of the plot.

   (c) The main computations take place in line 10, invoking `EGTMFDyn`.

      i. The input is: the payoff matrix `B`, the `'Rep1'` (continuous time replicator) dynamics, the initial state `s0` and the final time `J`.

      ii. The output is: `s` (the i-th line contains the population vector) at time $t(i)$, `x` (the i-th line contains the frequencies vector) at the $t(i)$, `t` (the vector fo times at which the ODE solver computes `s` and `x`).

   (d) In line 11 we use `GrDynX` to plot `x` vs. `t`.

6. In lines 13-16 we run the simulation of the evolutionary game.

   (a) Line 13: we will run `J` simulation steps which we take equal to 200. Note that simulation time is different from ODE integration time. Also, simulation steps are, by definition, equal, but ODE integration steps are, in general, unequal.

   (b) Line 14 is analogous to line 9.

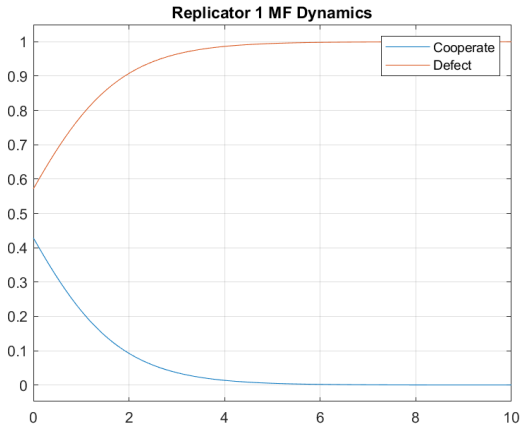
   (c) The main computations take place in line 10, invoking `EGTMFSim`.

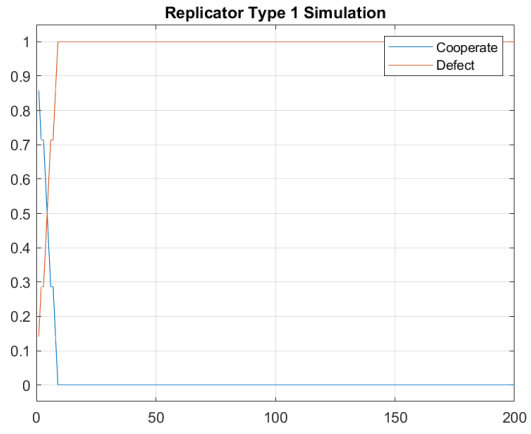
      i. The input is the same as for `EGTMFDyn`;

      ii. The output is the same as for `EGTMFDyn` except that there is no need for a times vector `t`.

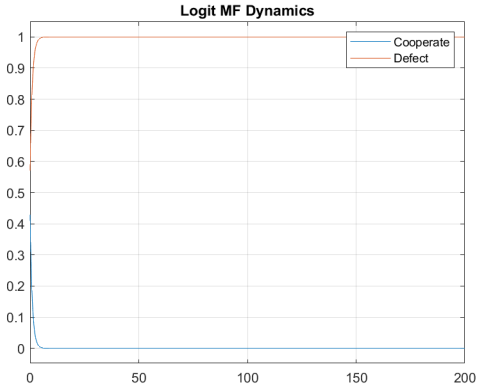
   (d) In line 11 we use `GrSim` to plot `x` vs. `t`.

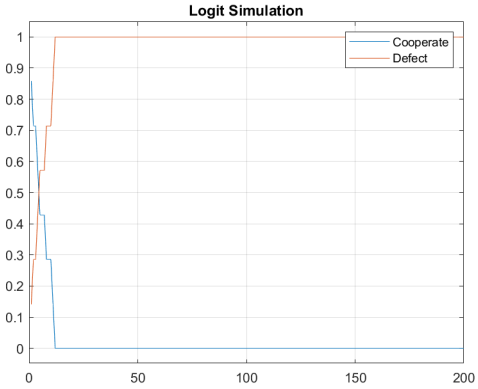
The results of the `PD_MFD01` are plotted and stored into the files `PD_MFDRep1.png`, `PD_SimRep1.png`, which are presented in the following figures.

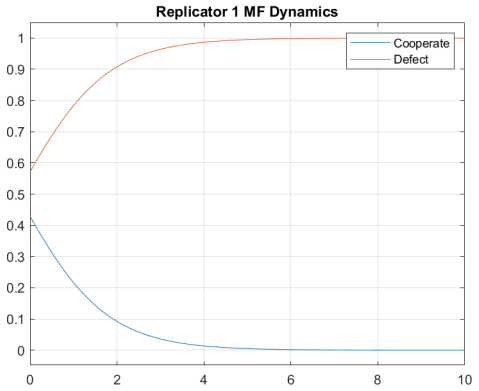




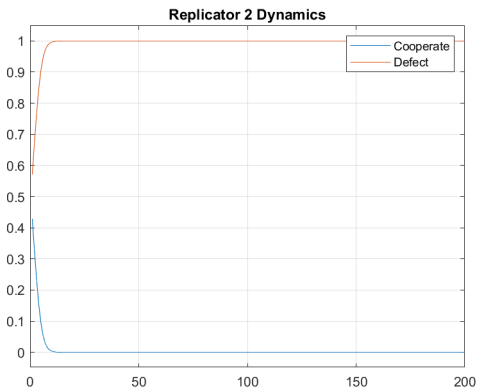
The Matlab scripts `PD_MFD02.m` and `PD_Sim02.m`, contained in `Examples/Basic`, repeat the MFD computation and the simulation for various dynamics. The results are plotted below.

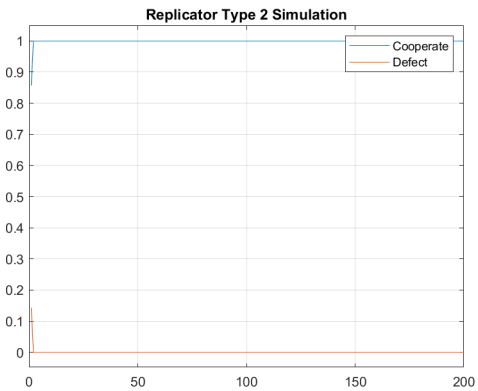

Logit MF Dynamics
Cooperate
Defect

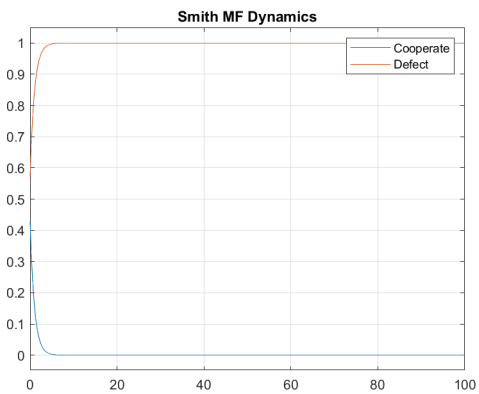

Logit Simulation
Cooperate
Defect

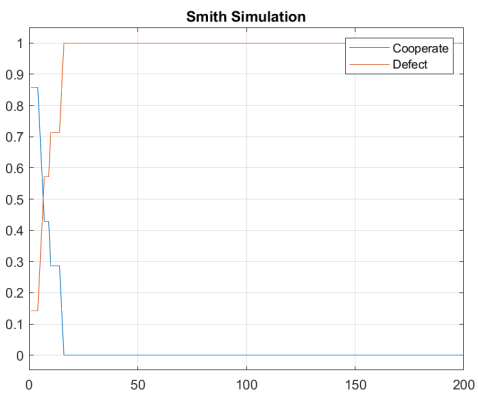

Replicator 1 MF Dynamics
Cooperate
Defect


Replicator Type 1 Simulation
Cooperate
Defect


Replicator 2 Dynamics
Cooperate
Defect


Replicator Type 2 Simulation
Cooperate
Defect


Smith MF Dynamics
Cooperate
Defect

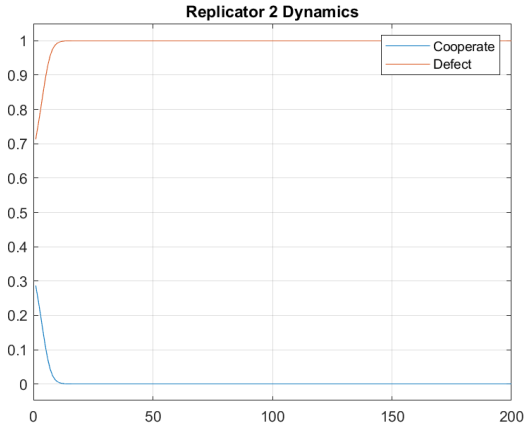

Smith Simulation
Cooperate
Defect

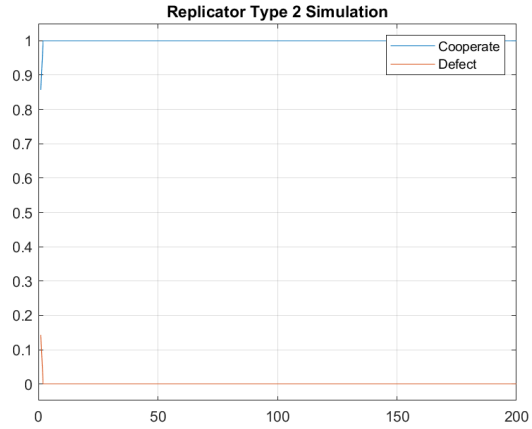
5.2 Stag Hunt

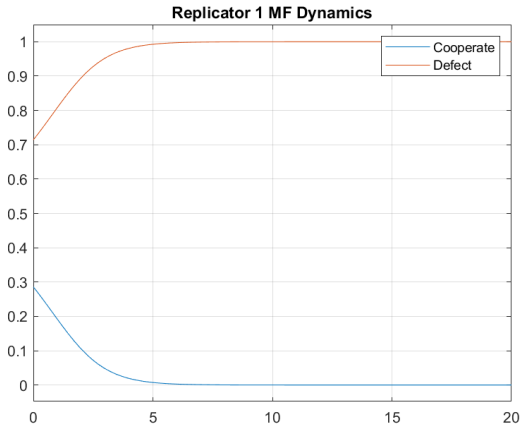
The Matlab scripts `SH_MFD.m` and `SH_Sim.m`, contained in `Examples/Basic`, repeat the MFD computation and the simulation for various dynamics, for an evolutionary game with the Stag Hunt as base game. This has the the payoff matrix

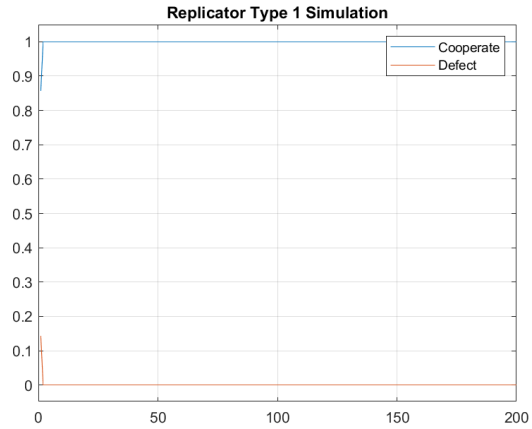
$$A = \begin{bmatrix} 4 & 1 \\ 3 & 2 \end{bmatrix}$$

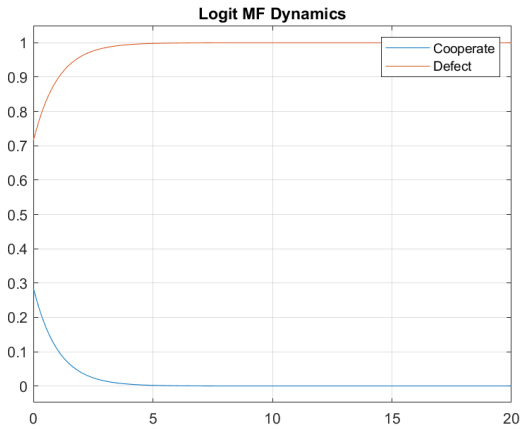
The structure of the scripts as very similar to that of the previously discussed `PD_MFD.m` and `PD_Sim.m`. The results are plotted below.

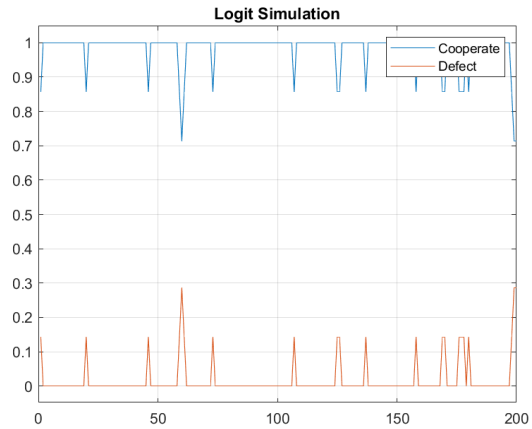


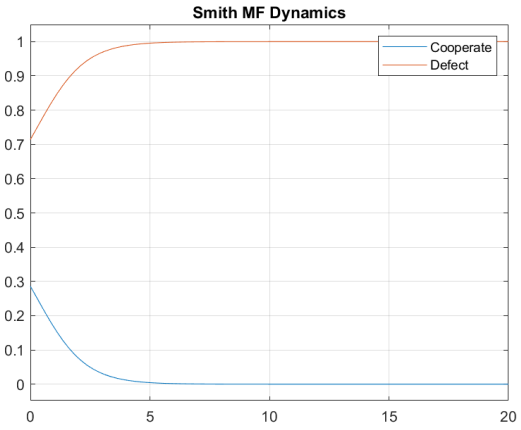


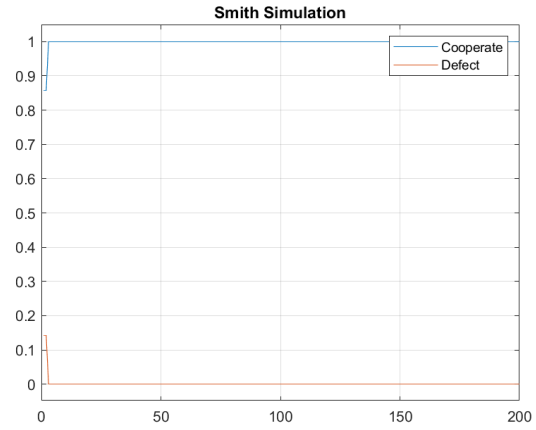










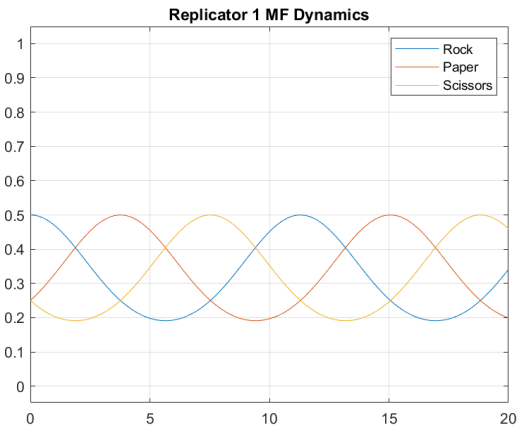


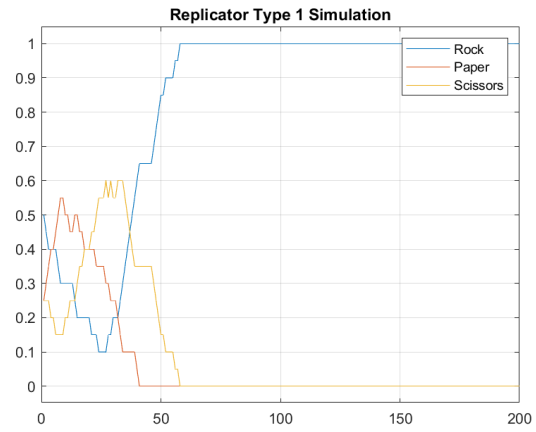
5.3 Rock-Paper-Scissors

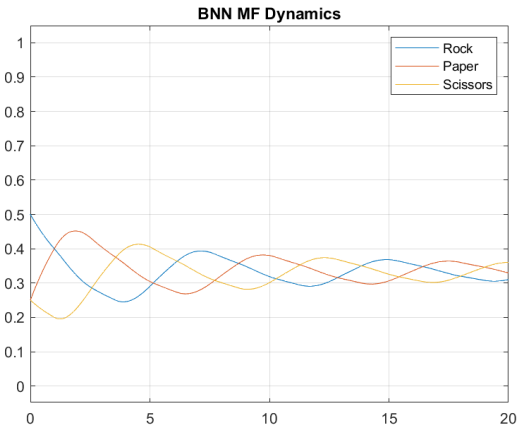
The Matlab scripts `RPS_MFD.m` and `RPS_Sim.m`, contained in `Examples/Basic`, repeat the MFD computation and the simulation for various dynamics, for an evolutionary game with the Rock-Paper-Scissors as base game. This has the the payoff matrix

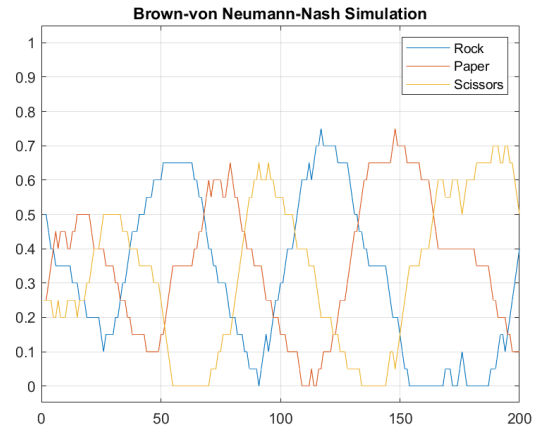
$$A = \begin{bmatrix} 0 & -1 & 1 \\ 1 & 0 & -1 \\ -1 & 1 & 0 \end{bmatrix}$$

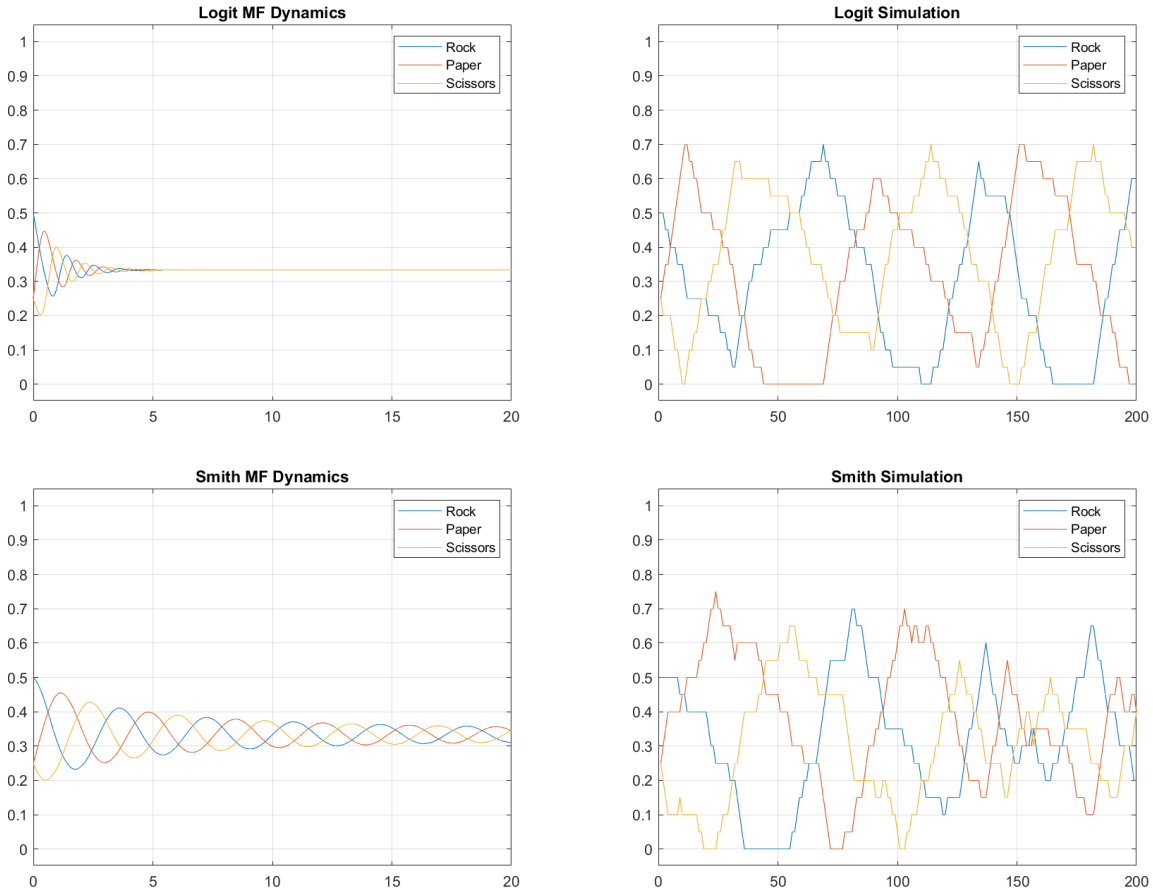
The structure of the scripts as very similar to that of the previously discussed `PD_MFD.m` and `PD_Sim.m`. The results are plotted below.



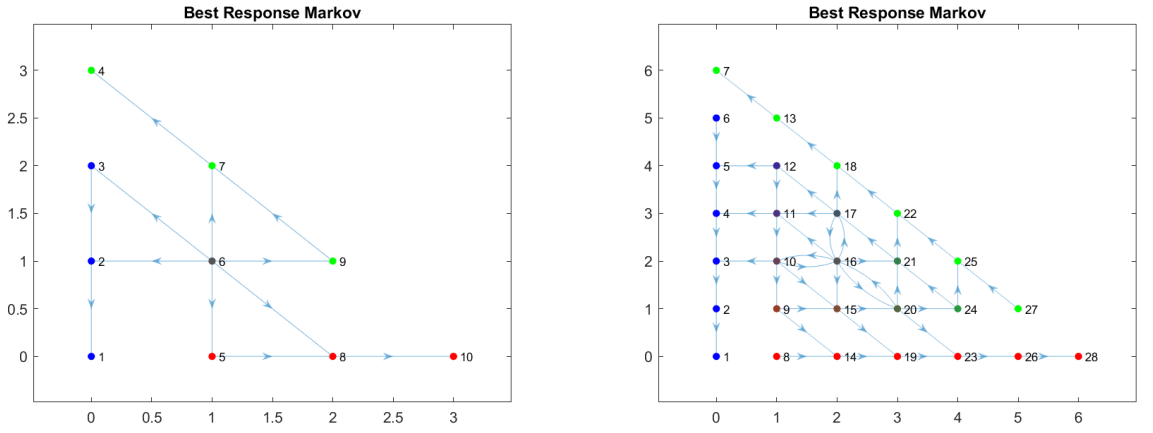




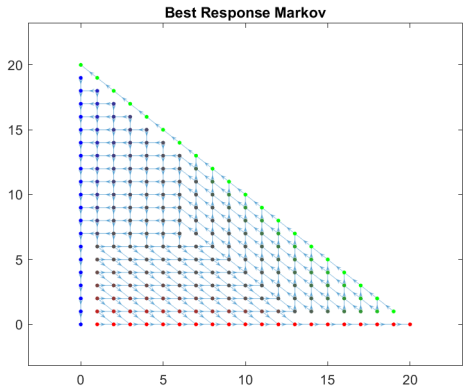


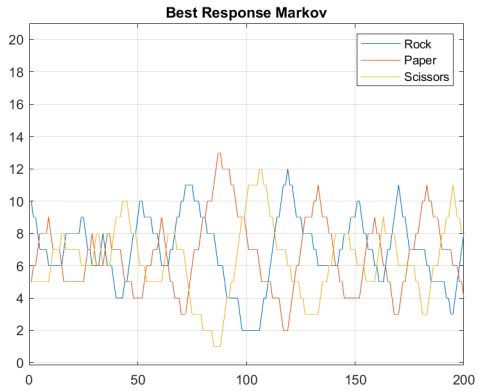


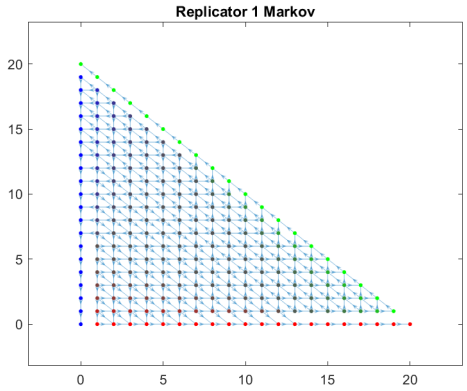
In addition we can use the transition probabilty matrix P to plot the state transition graphs (STGs). The Matlab scripts `RPS_Mrk01.m` and `RPS_Mrk02.m` (contained in the folder `Examples/Basic`) implement this for $N = 3$ and $N = 6$ players respectively; the results are plotted in the following two figures. Note the symmetric nature of the diagrams.

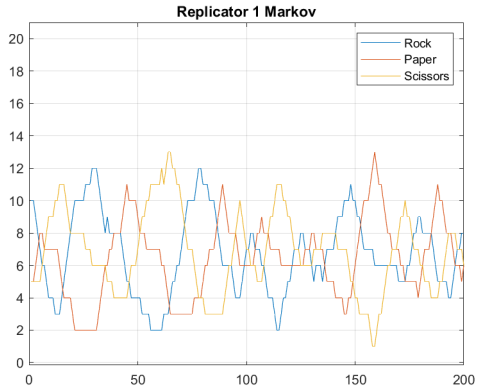


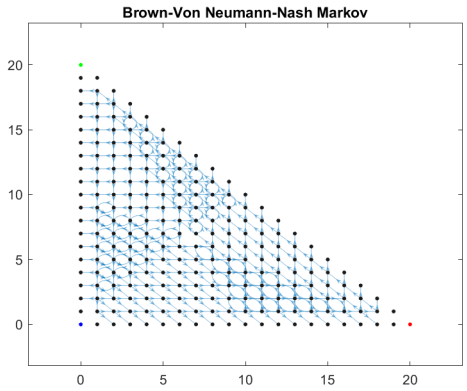
We repeat the Markovian analysis with additional revision protocols. The results are plotted in the following figures. In each figure pair, the left figure shows the STG for $N = 20$ players and the right figure shows a Markovian simulation with initial state $\mathbf{s}_0 = (10, 5, 5)$.

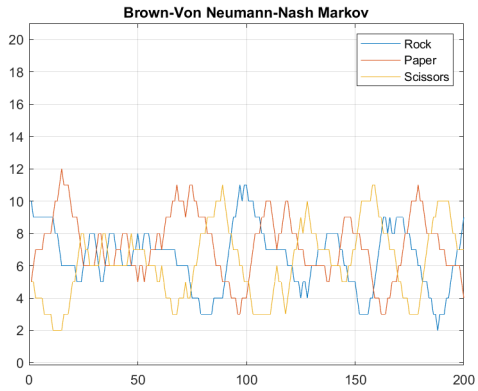

Best Response Markov

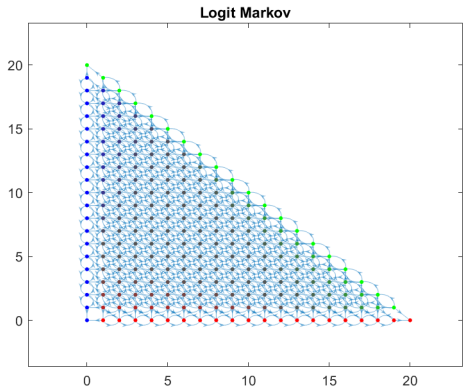

Best Response Markov
Rock
Paper
Scissors

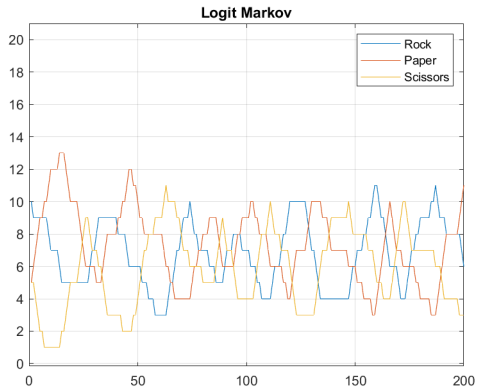

Replicator 1 Markov

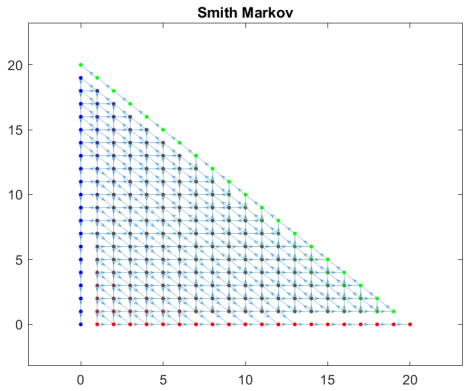

Replicator 1 Markov
Rock
Paper
Scissors

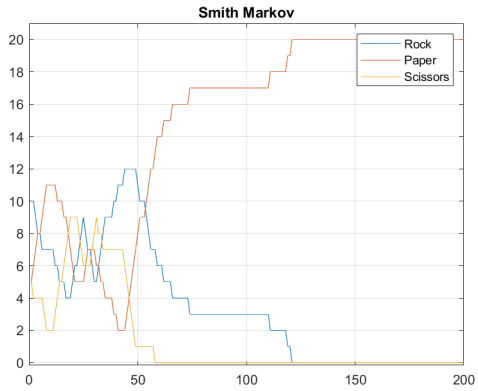

Brown-Von Neumann-Nash Markov


Brown-Von Neumann-Nash Markov
Rock
Paper
Scissors


Logit Markov


Logit Markov
Rock
Paper
Scissors





5.4 Iterated Prisoner's Dilemma

We will now study an evolutionary game obtained from an iterated two-player *base game*. The base game we use is the Prisoner's Dilemma [1] with bimatrix

$$A = \begin{bmatrix} 3,3 & 1,4 \\ 4,1 & 2,2 \end{bmatrix},$$

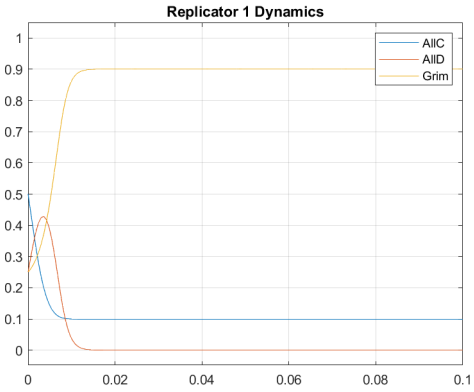
and we will use three strategies: $\sigma_1 =$"AllC" (always cooperate), $\sigma_2 =$"AllD" (always defect) and $\sigma_3 =$"TitForTat". From A we construct an $M \times M$ matrix B where

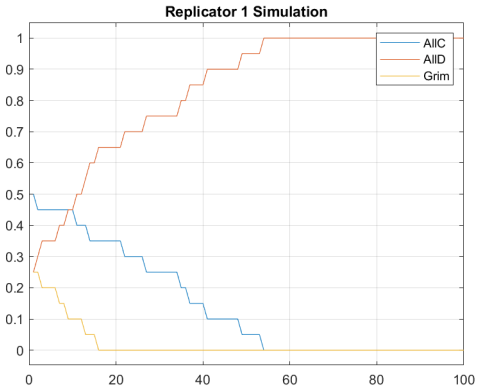
$$B_{m_1 m_2} = \text{"the payoff received by playing } T \text{ rounds of } A\text{, using strategy } \sigma_{m_1} \text{ against strategy } \sigma_{m_2}\text{"}.$$

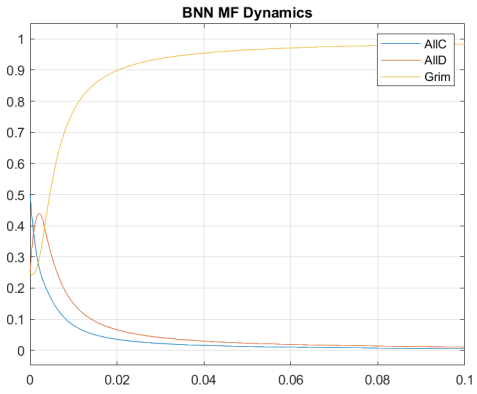
Further, we assume that the iterated game is played for $T = 1000$ rounds. Consider B_{11}, the payoff of a player who uses "AllC" for 1000 rounds, against another player who also uses "AllC"; hence the first player will receive $B_{11} = 1000 \cdot 3 = 3000$. Similarly, B_{12} is the payoff of a player who uses "AllC" for 1000 rounds, against another player who uses "AllD"; hence the first player will receive $B_{11} = 1000 \cdot 1 = 1000$. Continuing in this manner we obtain

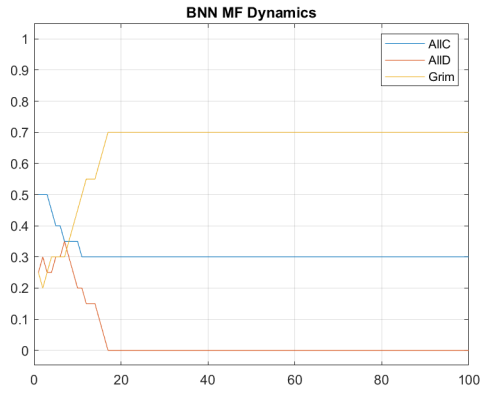
$$B = \begin{bmatrix} 3000 & 1000 & 3000 \\ 4000 & 2000 & 2002 \\ 3000 & 1999 & 3000 \end{bmatrix} \tag{5.1}$$

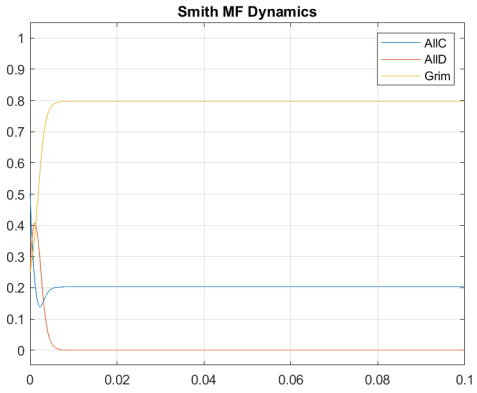
Now we compute the MFD dynamics and the plain simulation of the game (5.1) for starting state $\mathbf{s}_0 = (10, 5, 5)$ and various revision protocols; the strategies used are All-C, All-D and TitForTat. The experiments are coded in the files `IPD_MFD.m` and `IPD_Sim.m` (contained in `Examples/IterGms`) and the results are plotted in the following figures.

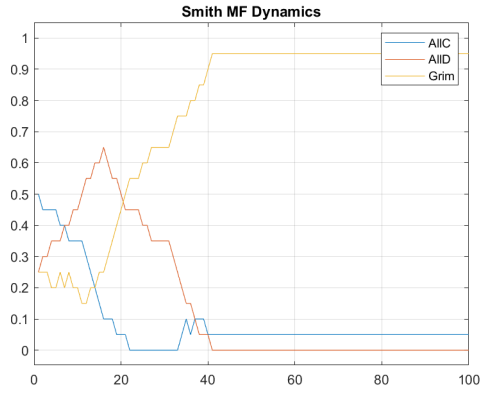


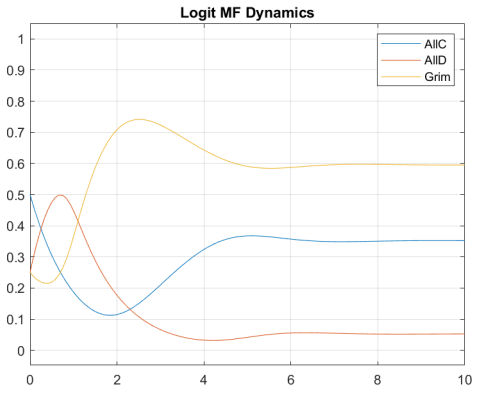


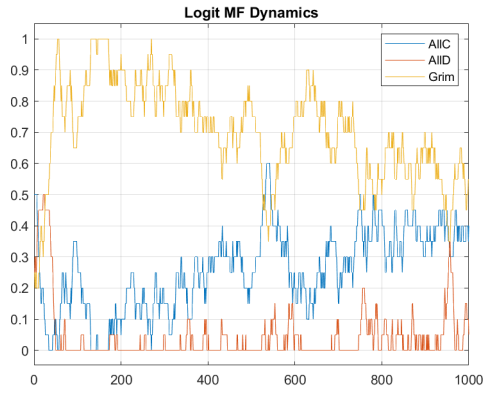




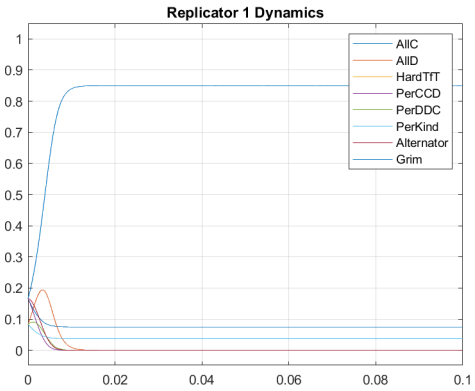


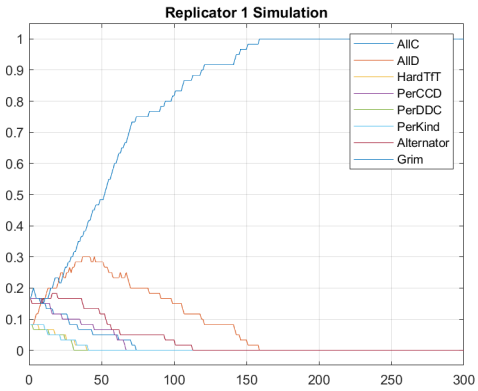


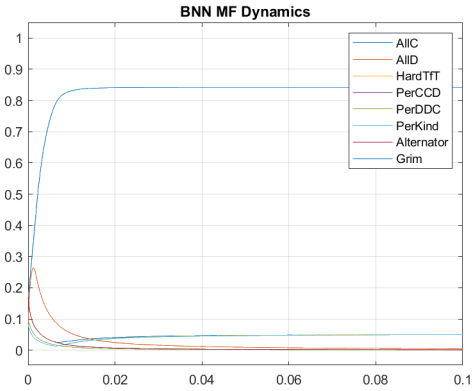


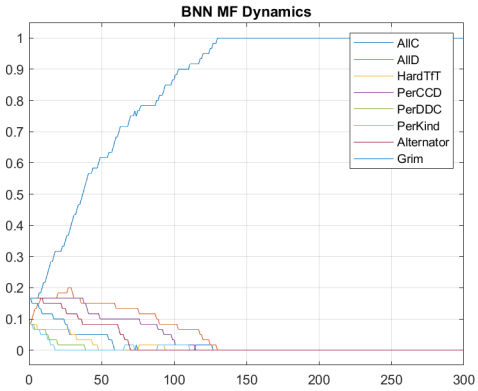


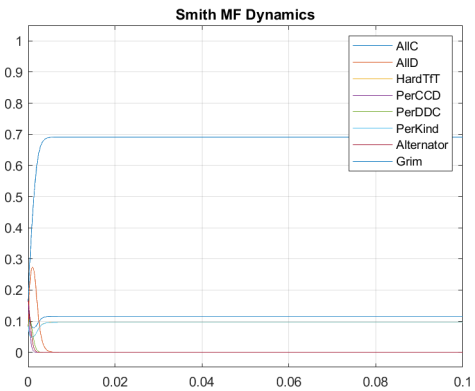
Next we repeat the same experiment with eight strategies; all other experimental parameters are the same. The results appear in the following figures.

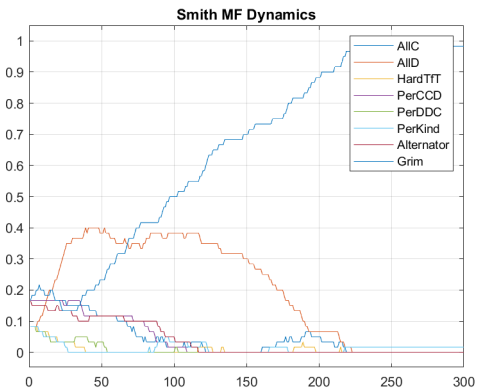

Replicator 1 Dynamics
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim

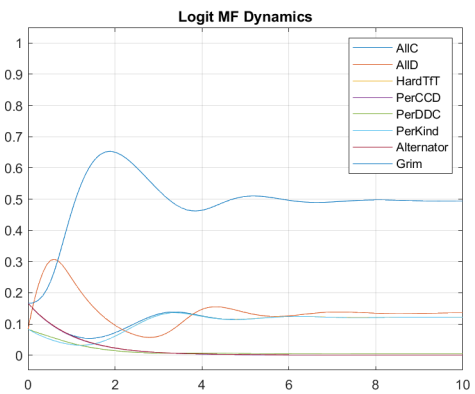

Replicator 1 Simulation
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim

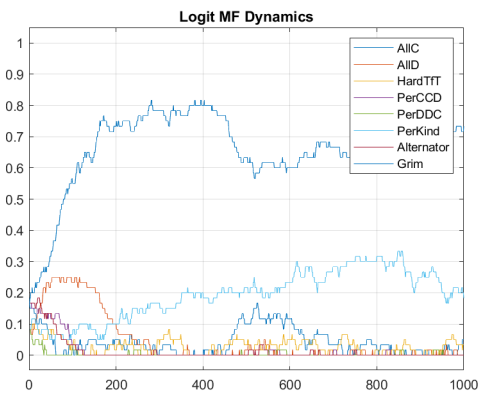

BNN MF Dynamics
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim


BNN MF Dynamics
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim

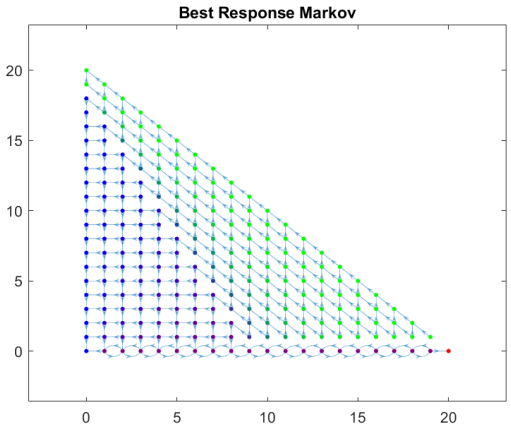

Smith MF Dynamics
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim

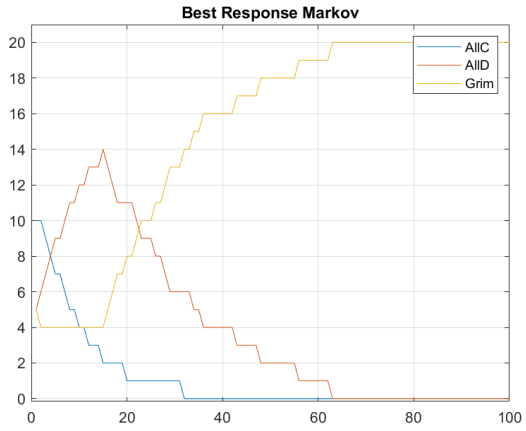

Smith MF Dynamics
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim


Logit MF Dynamics
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim

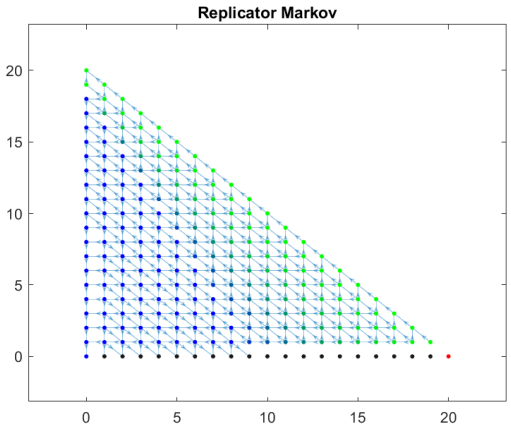

Logit MF Dynamics
AllC
AllD
HardTfT
PerCCD
PerDDC
PerKind
Alternator
Grim

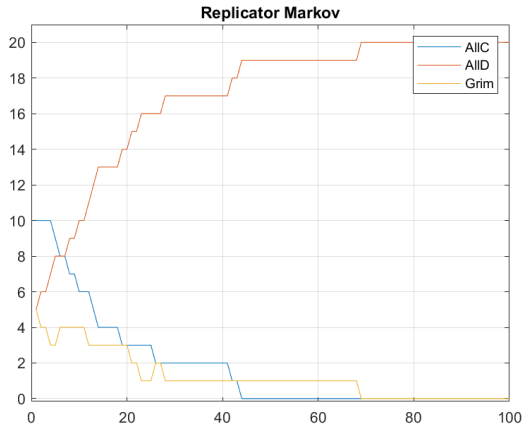
Now we perform the Markovian analysis of the same game for the same $\mathbf{s}_0 = (10, 5, 5)$ and the best response revision protocol. The experiments are coded in the file `IPD_Mrk.m.m` and the results are plotted in the following figures.

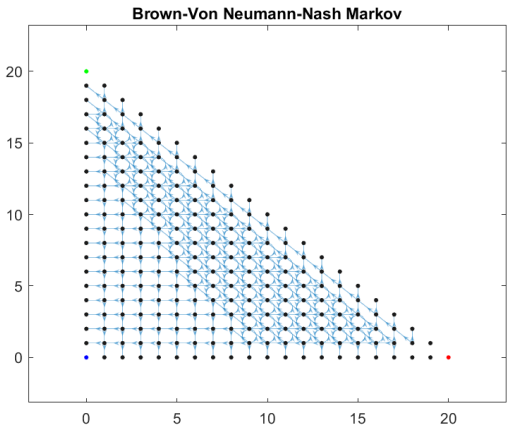


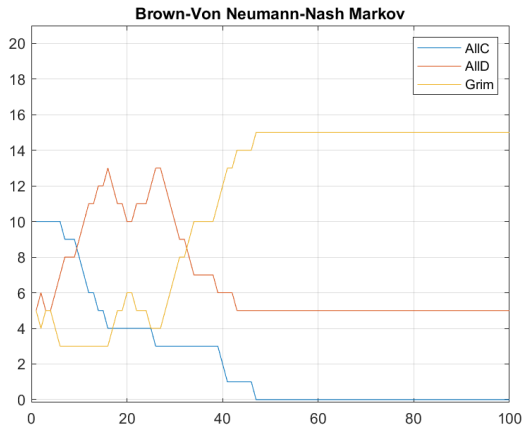


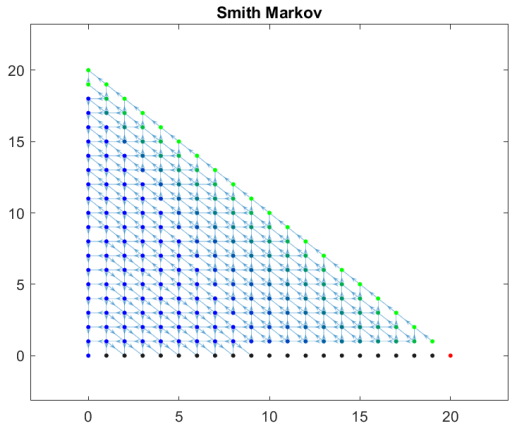
Finally, we repeat the above experiment with various revision protocols; the results are plotted below.

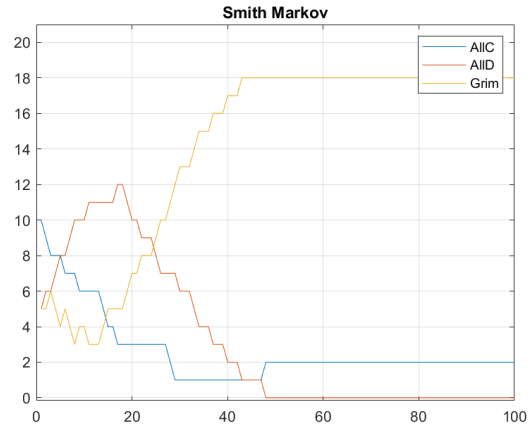


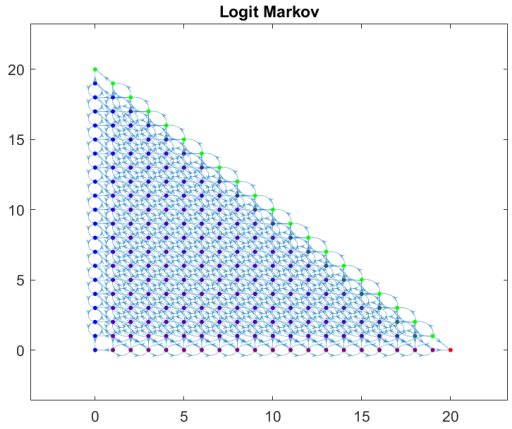


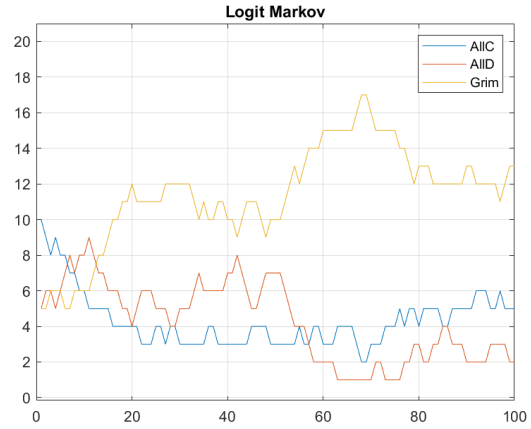












References

[1] R. Axelrod and W. D. Hamilton. "The evolution of cooperation." *Science*, vol. 211, pp. 1390-1396, 1981.

[2] J. Hofbauer and K. Sigmund. *Evolutionary games and population dynamics*. Cambridge University Press, 1998.

[3] J.G. Kemeny and J. L. Snell. *Finite markov chains*. Princeton, NJ: van Nostrand, 1969.

[4] P. Mathieu, B. Beaufils and J.P. Delahaye. "Studies on Dynamics in the Classical Iterated Prisoner's Dilemma with Few Strategies". In: Fonlupt, C., Hao, JK., Lutton, E., Schoenauer, M., Ronald, E. (eds) *Artificial Evolution. AE 1999. Lecture Notes in Computer Science*, vol. 1829. Springer, Berlin, Heidelberg, 1999.

[5] W.H. Sandholm, *Population Games and Evolutionary Dynamics*. MIT Press, 2010.

[6] W.H. Sandholm. "Local stability under evolutionary game dynamics". *Theoretical Economics*, vol. 5, pp. 27-50, 2010.

A Function Reference

The following is a list of the EGT functions and their intended use. Syntax details (input and output variables etc.) is only given for the main functions; for the rest of them, look at the corresponding Matlab files of the same name, which contain full description of the syntax.

A.1 Main Functions

A.1.1 Basic

EGT contains five basic functions:

1. `[P,S]=EGTMarkP(B,Dynamic)` computes the transition probability matrix of an evolutionary game.
2. `[s,x]=EGTMarkSim(P,S,s0,J)` simulates an evolutionary game using the transition probability matrix.
3. `[s,x,t]=EGTMFDyn(B,Dynamic,s0,J)` computes the the continuous time mean field dynamics of an evolutionary game.
4. `[s,x]=EGTMFDynDisc(B,Dynamic,s0,J)` computes the discrete time mean field dynamics of an evolutionary game.
5. `[s,x]=EGTSim(B,Dynamic,s0,J)` simulates an evolutionary game.

The input variables are as follows.

`B`	:	Payoff matrix
`Dynamic`	:	Dynamic or Protocol
`P`	:	Transition probability matrix
`S`	:	Srategy state space matrix
`s0`	:	Initial state
`J`	:	Number of generations

In addition the following variables essentially work as input to the main functions, but must be declared as global (for more details see the script listings in the Examples section).

`B`	:	Payoff matrix
`M`	:	Number of strategies
`N`	:	Number of players
`eta`	:	Noise parameter (only used by Logit functions)

The output variables are as follows.

`P`	:	Transition probability matrix
`S`	:	Srategy state space matrix
`s`	:	Strategy state sequence (across generations)
`x`	:	Frequency sequence (across generations)
`t`	:	Times sequence

The variable `t` is needed because the Matlab ODE solvers do not always output the vector of $x(t)$ values at equispaced time steps.

A.1.2 Dynamics

These functions are used by `EGTMFDyn` and `EGTMFDynDisc`. They all work similarly to provide the update term for the various dynamics. For details of their syntax look inside the corresponding `*.m` files.

Function	Dynamics
`DynBNN`	Brown-von Neumann-Nash
`DynLogit`	Logit
`DynRep1`	Replicator Type 1
`DynRep2`	Replicator Type 2
`DynSmith`	Smith

A.1.3 Rate Matrices

These functions are used by `EGTMarkP`. They all work similarly to provide the the rate matrix R for the various protocols. For details of their syntax look inside the corresponding `*.m` files.

Function	Protocol
`MakeRateBNN`	Brown-von Neumann-Nash
`MakeRateBR`	Logit
`MakeRateLogit`	Replicator Type 1
`MakeRateRep1`	Replicator Type 2
`MakeRateSmith`	Smith

A.1.4 Simulation

These functions are used by `EGTSim`. They all work similarly to to simulate a generation update for the various dynamics. For details of their syntax look inside the corresponding `*.m` files.

Function	Protocol
`SimBNN`	Brown-von Neumann-Nash
`SimBR`	Best Response
`SimLogit`	Logit
`SimRep1`	Replicator Type 1
`SimRep2`	Replicator Type 2
`SimSmith`	Smith

A.2 Auxiliary Functions

A.2.1 Plotting

These functions are used for plotting. For details of their syntax look inside the corresponding `*.m` files.

`GrDyn`	Plots a sequence of frequency vectors $\mathbf{x}(1)$, ..., $\mathbf{x}(J)$, generated by `EGTMFDynDisc` (discrete time).
`GrDynX`	Plots the frequency function $\mathbf{x}(t)$ at times $\mathbf{t}$, generated by `EGTMFDyn` (continuous time).
`GrSim`	Plots a sequence of generation vectors $\mathbf{s}(1)$, ..., $\mathbf{s}(J)$, generated by `EGTSim`.
`GrMrk`	Plots the State Transition Graph (STG) of a Markov chain with transition matrix P as well as the plot of a strategy state sequence produced by the same P. Only works for three strategies.
`GrSimplex`	Plots a sequence of frequency vectors $\mathbf{x}(1)$, ..., $\mathbf{x}(J)$, generated by `EGTMFDyn`, on the 3-simplex. Only works for three strategies.

A.2.2 Creation of Auxiliary Quantities

These create various quantities which are required by the main functions. For details of their syntax look inside the corresponding `*.m` files.

`MakeNumState`	Given a state $\mathbf{s}$, it computes a standardized number. Only works for three strategies.
`MakePayoff`	Computes the payoff matrix B.
`MakeQ`	Computes the strategy payoff vector $\mathbf{Q}(\mathbf{s})$, for all $\mathbf{s} \in S$.
`MakeS`	Creates the state space S as a matrix. Only works for three strategies.
`MakeSfromZ`	Computes a state vector $\mathbf{s}$ from a population vector $\mathbf{z}$. Only works for three strategies.
`MakeZfromS`	Computes a population vector $\mathbf{z}$ from a state vector $\mathbf{s}$. Only works for three strategies.

A.3 Strategy Functions

Given an iterated game history, these functions compute a player's next move when the he uses the strategy implied by the function name (and explained in the respective files). The strategies have been copied from [4]; some are well known, others not. We only give here a partial list, of the better known strategies.

`AllC`	Always cooperate
`AllD`	Always defect
`Grim`	Cooperate in every round, until the opponent defects and then defect in all subsequent rounds
`Pavlov`	When the player and his opponent play the same (resp. different) move the player defects (resp. cooperates) in the next round
`TitForTat`	Cooperates in the first round, then always plays the opponent's previous move

It is relatively easy to write your own strategies. All strategy files follow the same input / output convention; just open one of them, rename it and change the logic, to implement your own strategy.

B The GameTheory2025 Collective

The origin of this toolbox was a team project assigned to the students of the Game Theory class taught in Spring 2025, at the Department of Electrical and Computer Engineering, Aristotle University of Thessaloniki, Greece. Eight teams contributed their versions of the toolbox. Their members form the "*GameTheory2025 Collective*" and they are the following:

Team No.	**Members**
0	Ath. Kehagias (Professor)
1	I. Lazaridis, I. Vardakis, K. Vlahakos
2	E. Delopoulos, D. Vaporis, V. Vlasios
3	G. Daios, G. Mousses, S. Nazlidis, K. Papadakis
4	T. Badas, G. Binos, G. Hatzilyras
5	N. Gaitanidis, D. Moutaftsidis, N. Stamatis
6	T. Billas, D. Kokkinakos, H. Marinopoulos
7	G. Gioulekas, E. Koumparidou, N. Kousta
8	I. Konstantakis, V. Kordis, I. Matsrogiannis

The Evolutionary Game Toolboxes created by each team and submitted in the class can be found on GitHub. They can be considered as preliminary versions of the current toolbox. The original contributions were reworked by Ath. Kehagias and evolved (pun intended) to the current version of EGT.

C EGT-similar Toolboxes

Several toolboxes / libraries comparable to EGT are available; each has its own strengths. Here is a partial list.

1. ABED. A NetLogo application. "ABED is free and open-source software for *simulating* evolutionary game dynamics in finite populations."

   Link: `https://luis-r-izquierdo.github.io/abed-1pop/`

2. Axelrod. A Python library. "A Python library with the goals of enabling the reproduction of previous Iterated Prisoner's Dilemma research as easily as possible."

   Link: `https://github.com/Axelrod-Python/Axelrod`

3. EGT Tools. A Python library. "EGTtools provides a centralized repository with analytical and numerical methods to study/model game theoretical problems under the Evolutionary Game Theory (EGT) framework."

   Link: `https://egttools.readthedocs.io/en/latest/`

4. Evolutionary Games. An R package. " A comprehensive set of tools to illustrate the core concepts of evolutionary game theory, such as evolutionary stability or various evolutionary dynamics."

   Link: `https://cran.r-project.org/web/packages/EvolutionaryGames/index.html`

5. ipd. It is a modern Python version of Prison. "It contains Python code and Jupyter notebooks to understand easily how to experiment computational game theory, build and evaluate strategies at the iterated prisoner's dilemma (IPD) and gives many tools for sophisticated experiments. It is also useful to reproduce most of the experiences of our research articles cited in the bibliography."

   Link: `https://github.com/cristal-smac/ipd/blob/master/README.md`

6. NashPy. A Python library. "A python library for 2 player games."

   Link: `https://nashpy.readthedocs.io/en/stable/`

7. PDToolbox. A Matlab toolbox. "PDToolbox is a matlab implementation of some evolutionary dynamics from game theory."

   Link: `https://github.com/carlobar/PDToolbox_matlab`

8. WinPri and Prison. These provided the original motivation for writing EGT. They are two forgotten gems, created in the 1990's by B. Beaufils, J.P. Delahaye and P. Mathieu. WinPri is a Windows 3.1 program and can only be run on an emulator (e.g., DosBox). Prison is written in ANSI C and can be compiled to run on modern Windows systems. I have not been able to find a live link for these programs.